

Floating Point

15-213: Introduction to Computer Systems
4th Lecture, Sep. 10, 2015

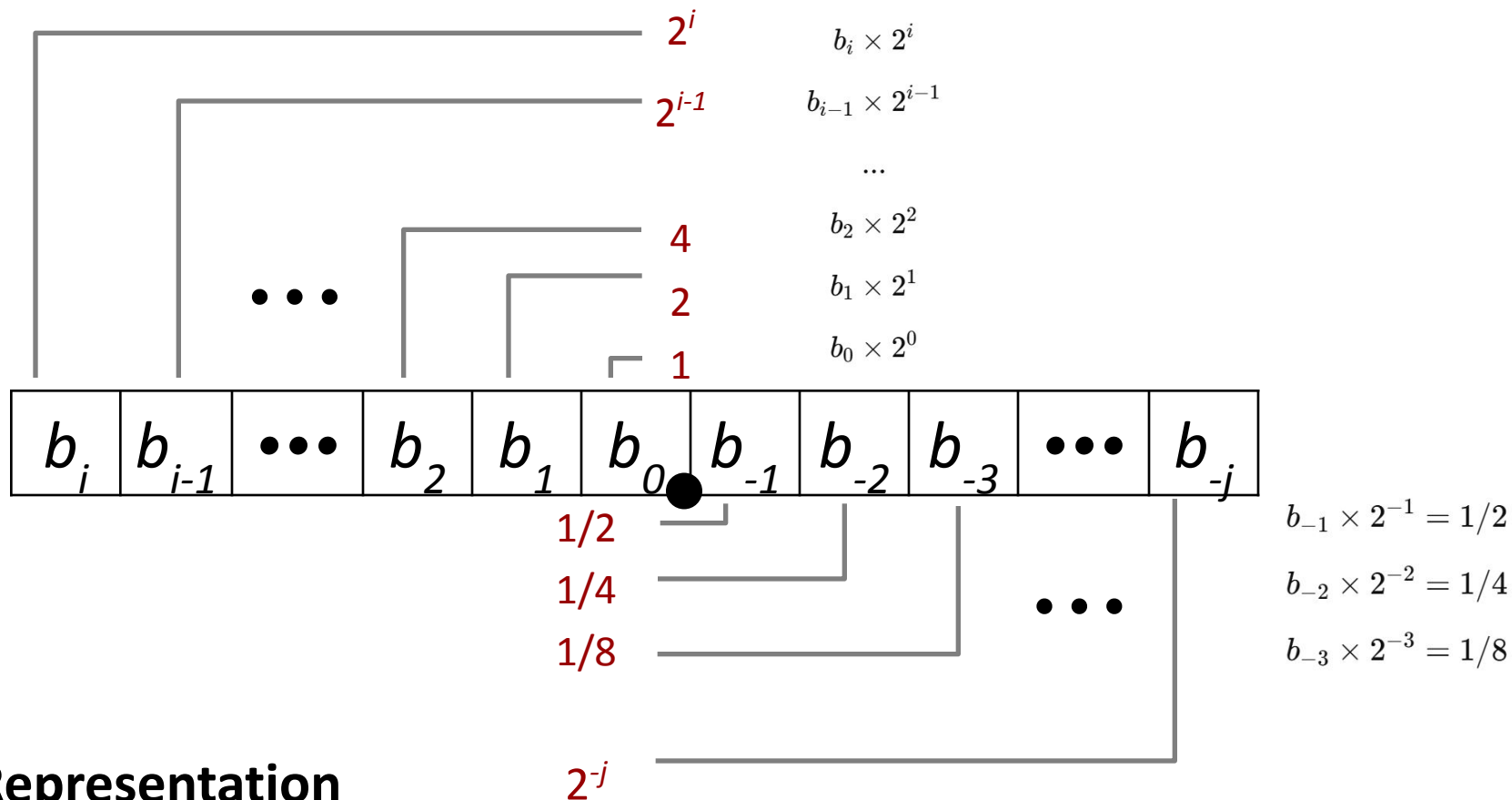
Instructors:

Randal E. Bryant and David R. O'Hallaron

Today: Floating Point

- **Background: Fractional binary numbers**
- **IEEE floating point standard: Definition**
- **Example and properties**
- **Rounding, addition, multiplication**
- **Floating point in C**
- **Summary**

Fractional Binary Numbers



■ Representation

- Bits to right of “binary point” represent **fractional powers of 2**
- Represents rational number:

$$\sum_{k=-j}^i b_k \times 2^k$$

Fractional binary numbers

■ What is 1011.101_2 ?

1. Split it:

- Integer part: **1011**
- Fractional part: **.101**

2. Expand Using Powers of 2:

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

⇐ Integer part: 8+2+1

$$+1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

⇐ Fractional part: 0.5 + 0 + 0.125

3. Final answer: **11.625**

Binary fractions use negative powers of 2.

Fractional Binary Numbers: Examples

■ Value Representation

5 3/4 101.11_2

2 7/8 10.111_2

1 7/16 1.0111_2

Binary numbers
exactly representable:

$$\frac{\text{integer}}{2^n}$$

■ Observations

- Divide by 2 by shifting right (unsigned)
- Multiply by 2 by shifting left
- Numbers of form $0.111111\dots_2$ are just below 1.0
 - $1/2 + 1/4 + 1/8 + \dots + 1/2^i + \dots \rightarrow 1.0$
 - Use notation $1.0 - \epsilon$

Representable Numbers

■ Limitation #1

- Can only exactly represent numbers of the form $x/2^k$
 - Other rational numbers have repeating bit representations
- Value Representation
 - $1/3$ $0.0101010101[01]\dots_2$
 - $1/5$ $0.001100110011[0011]\dots_2$
 - $1/10$ $0.0001100110011[0011]\dots_2$

$0.333\dots$,
 0.2 ,
 0.1
 are NOT exact in binary.

■ Limitation #2

- Just one setting of binary point within the w bits
 - Limited range of numbers (very small values? very large?)

Example: With 3 fractional bits

Possible fractions: Only 8 possible values between 0 and 1.

0.000
0.001
0.010
0.011
0.100
0.101
0.110
0.111

Some numbers are representable

Some are impossible

Some are approximated

demo: ch2_float.c

Today: Floating Point

- Background: Fractional binary numbers
- **IEEE floating point standard: Definition**
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

IEEE Floating Point

Floating point is not designed for simplicity.
It is designed for numerical reliability.

■ IEEE Standard 754

- Established in 1985 as uniform standard for floating point arithmetic
 - Before that, many idiosyncratic formats
- Supported by all major CPUs
 - Stability
 - Correct rounding
 - Error behavior
 - Reproducibility

■ Driven by numerical concerns

- Nice standards for rounding, overflow, underflow
- Hard to make fast in hardware
 - Numerical analysts predominated over hardware designers in defining standard

Floating Point Representation

■ Numerical Form:

$$(-1)^s M 2^E$$

- **Sign bit s** determines whether number is negative or positive
- **Significand M** normally a fractional value in range $[1.0, 2.0)$.
- **Exponent E** weights value by power of two

$$1.0 \leq M < 2.0$$

M is like the “decimal part” in scientific notation.

$$1.xxxxx \times 2^E$$

M

■ Encoding

- MSB s is sign bit s
- **exp** field encodes E (but is not equal to E)
- **frac** field encodes M (but is not equal to M)

$$1.0111 \times 2^2$$

$$s = 0$$

$$E = 2$$

$$M = 0.0111$$



Precision options

Floating point cannot use infinite precision. Memory and hardware are finite. FP is a compromise.

Sign → positive or negative

Exponent → range

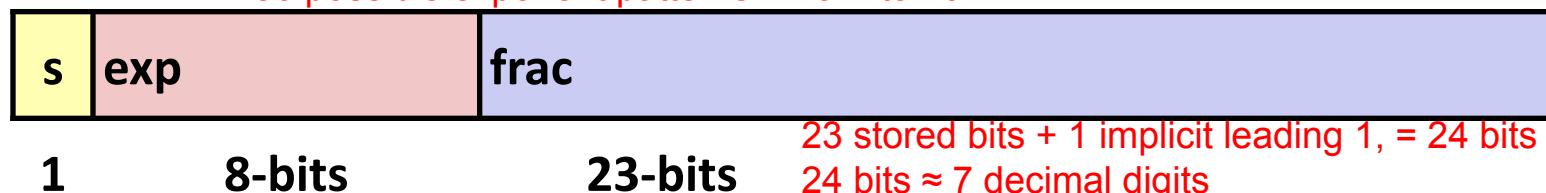
Fraction → precision

More exponent bits → bigger range

More fraction bits → more precision

■ Single precision: 32 bits

$2^8 = 256$ possible exponent patterns. $\sim 10^{-38}$ to 10^{38}



23 stored bits + 1 implicit leading 1, = 24 bits of precision
 24 bits $\approx$ 7 decimal digits
 So single precision gives about:
 ~ 7 reliable decimal digits.

■ Double precision: 64 bits



53 bits $\approx$ 15–16 decimal digits.
 double is default in C.

■ Extended precision: 80 bits (Intel only)



“Normalized” Values

$$v = (-1)^s M 2^E$$

- **When: $\text{exp} \neq 000\dots 0$ and $\text{exp} \neq 111\dots 1$**
 - $\text{exp} = 000\dots 0 \rightarrow$ special case (denormal / zero)
 - $\text{exp} = 111\dots 1 \rightarrow$ special case (∞ or NaN)

- **Exponent coded as a *biased* value: $E = \text{Exp} - \text{Bias}$**
 - Exp : **unsigned** value of exp field
 - $\text{Bias} = 2^{k-1} - 1$, where k is number of exponent bits
 - Single precision: 127 (Exp: 1...254, **E: -126...127**) 10^{-38} to 10^{38}
 - Double precision: 1023 (Exp: 1...2046, E: -1022...1023)

- **Significand coded with implied leading 1: $M = 1.\text{xxx}\dots\text{x}_2$**
 - xxx...x: bits of frac field
 - Minimum when frac=000...0 ($M = 1.0$)
 - Maximum when frac=111...1 ($M = 2.0 - \epsilon$)
 - Get extra leading bit for “free”

Exponent field = 8 bits

Bias = $2^7 - 1 = 127$

If Exp field stores:

127 $\rightarrow$ actual E = 0

128 $\rightarrow$ actual E = 1

126 $\rightarrow$ actual E = -1

we assume 1, and do not store the leading 1

Normalized Encoding Example

$$v = (-1)^s M 2^E$$

$$E = \text{Exp} - \text{Bias}$$

$$\text{Exp} = E + \text{Bias}$$

■ Value: float $F = 15213.0$;

$$15213_{10} = 11101101101101_2$$

$$= 1.1101101101101_2 \times 2^{13}$$

Convert 15213 to binary.

we shifted the point 13 places left.

■ Significand

$$M = 1.\underline{1101101101101}_2$$

$$\text{frac} = \underline{1101101101101}0000000000_2$$

IEEE does not store the leading 1.
Then pad zeros to reach 23 bits:

■ Exponent

$$E = 13$$

$$\text{Bias} = 127$$

$$\text{Exp} = 140 = 10001100_2$$

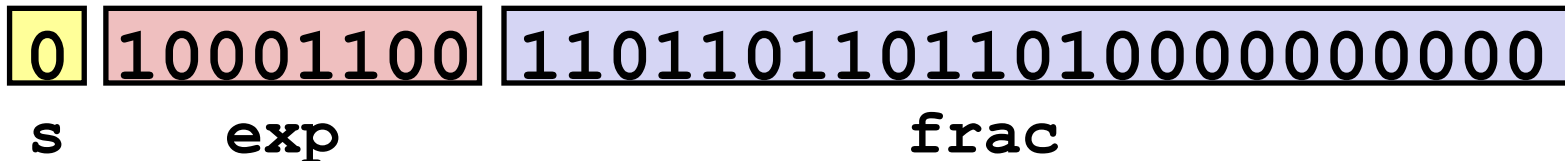
$$\text{Exp} = E + \text{Bias}$$

$$= 13 + 127 = 140$$

$$v = (-1)^s M 2^E$$

$$v = 1.1101101101101 \times 2^{13}$$

■ Result:



IEEE Encoding Procedure

1. Convert number to binary
2. Normalize to $1.x \times 2^E$
3. Sign bit = sign
4. Exponent field = $E + \text{bias}$
5. Fraction field = bits after decimal
6. Pad to required length

demo: ch2_float_15213.c:

program shows $M = 1.8570556640625$ (decimal)

Denormalized Values

$$v = (-1)^s M 2^E$$

$$E = 1 - \text{Bias}$$

- **Condition: $\text{exp} = 000\dots 0$**

smallest normal value 1.0×2^{-126}
 Without denormals $\rightarrow$ it would jump straight to 0.
 IEEE does not like sudden jumps.
 So it introduces **gradual underflow**.
 That's **denormals**.
- **Exponent value: $E = 1 - \text{Bias}$ (instead of $E = 0 - \text{Bias}$)** IEEE defines
- **Significand coded with implied **leading 0**: $M = 0.\text{xxx}\dots\text{x}_2$**

▪ $\text{xxx}\dots\text{x}$: bits of frac

■ Cases

Normalized numbers: $1.\text{xxx} \times 2^{126}$

Denormal numbers: $0.\text{xxx} \times 2^{126}$

- $\text{exp} = 000\dots 0, \text{frac} = 000\dots 0$
 - Represents **zero** value
 - Note distinct values: $+0$ and -0 (why?) sign bit still matters.

So they fill in the tiny numbers between: 0 and 1.0×2^{126} gradually.

That's the key idea.

- $\text{exp} = 000\dots 0, \text{frac} \neq 000\dots 0$ **denormal**

- Numbers closest to 0.0
- Equispaced: **Every denormal number increases by exactly that amount.** 2^{-149}

Smallest denormal $M \cdot E$:
 $2^{-23} \times 2^{-126} = 2^{-149}$

Comparison

Case	exp	M	E
Normal	$\neq 0, \neq \text{all } 1\text{s}$	1.xxx	Exp – Bias
Denormal	0	0.xxx	1 – Bias
Zero	0	0	0
Inf/NaN	all 1s	special	special

Special Values

demo: ch2_inf.c:

■ Condition: `exp = 111...1`

■ Case: `exp = 111...1, frac = 000...0`

- Represents value ∞ (infinity)
- Operation that overflows
- Both positive and negative depending on sign bit.
- E.g., $1.0/0.0 = -1.0/-0.0 = +\infty$, $1.0/-0.0 = -\infty$

■ Case: `exp = 111...1, frac $\neq$ 000...0`

- Not-a-Number (NaN)
- Represents case when no numeric value can be determined
- E.g., $\text{sqrt}(-1)$, $\infty - \infty$, $\infty \times 0$

NaN

$$\sqrt{-1}$$

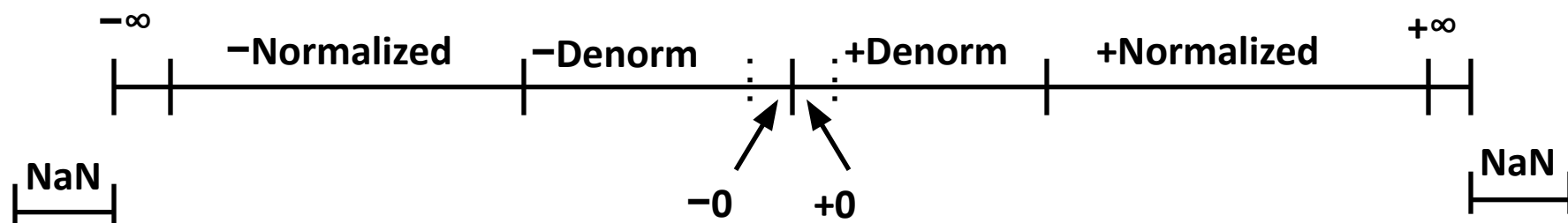
$$\infty - \infty$$

$$0/0$$

$$\infty \times 0$$

$+\infty$ and $-\infty$ preserves directional information.

Visualization: Floating Point Encodings



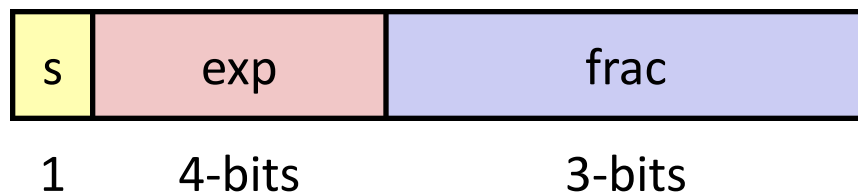
0 ---- denormals ---- smallest normal ---- larger normals

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- **Example and properties**
- Rounding, addition, multiplication
- Floating point in C
- Summary

Tiny Floating Point Example

We are building a mini IEEE.



Same rules. Smaller numbers.

$$\text{Bias} = 2^{k-1} - 1 = 2^3 - 1 = 7$$

■ 8-bit Floating Point Representation

- the sign bit is in the most significant bit
- the next four bits are the exponent, with a bias of 7
- the last three bits are the **frac**

normal: $v = (-1)^s (1.xxx) 2^E$

■ Same general form as IEEE Format **denormal:** $v = (-1)^s (0.xxx) 2^{1-7}$

- normalized, denormalized
- representation of 0, NaN, infinity

$$v = (-1)^s \times (0.xxx) \times 2^{-6}$$

infinity $+\infty$: 0 1111 000

Dynamic Range (Positive Only)

$$v = (-1)^s M 2^E$$

n: $E = \text{Exp} - \text{Bias}$
d: $E = 1 - \text{Bias}$

closest to zero

$k=4$, $\text{Bias}=2^{k-1}-1 = 2^3-1=7$
 $E = -6$ for denormal, fixed.

largest denorm

smallest norm

closest to 1 below

closest to 1 above

largest norm

Equispaced

Denormalized numbers

s	exp	frac	E	Value
0	0000	000	-6	0 $2^{-6}=1/64$
0	0000	001	-6	$(0+0+1/8) * 1/64 = 1/512$
0	0000	010	-6	$(0+1/4+0) * 1/64 = 2/512$
...				
0	0000	110	-6	$(1*1/2+1*1/4+0) * 1/64=6/512$
0	0000	111	-6	$(1*1/2+1*1/4+1*1/8) * 1/64=7/512$

0	0001	000	-6	$8/8*1/64 = 8/512$
0	0001	001	-6	$9/8*1/64 = 9/512$
...				

0	0110	110	-1	$14/8*1/2 = 14/16$
---	------	-----	----	--------------------

0	0110	111	-1	$15/8*1/2 = 15/16$ $(15/8=1+7/8)$
---	------	-----	----	---

0	0111	000	0	$8/8*1 = 1$
---	------	-----	---	-------------

0	0111	001	0	$9/8*1 = 9/8$
---	------	-----	---	---------------

0	0111	010	0	$10/8*1 = 10/8$
---	------	-----	---	-----------------

... $2^7=128$

0	1110	110	7	$14/8*128 = 224$
---	------	-----	---	------------------

0	1110	111	7	$15/8*128 = 240$
---	------	-----	---	------------------

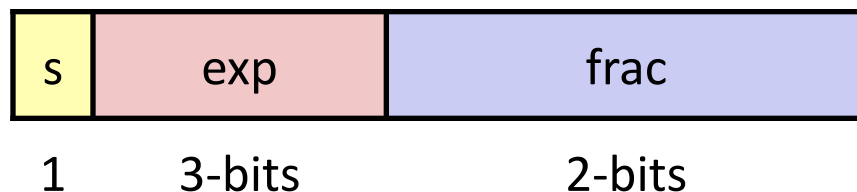
0	1111	000	n/a	inf
---	------	-----	-----	-----

demo: ch2_256.c

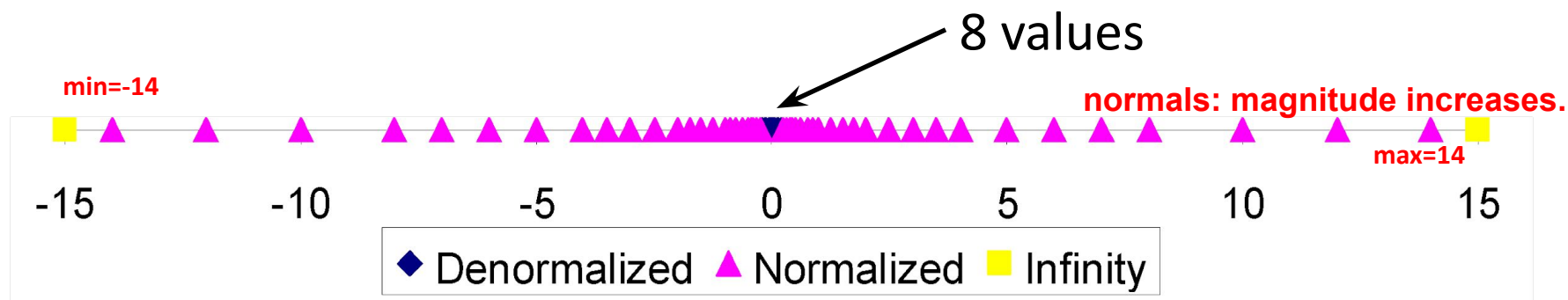
Distribution of Values

■ 6-bit IEEE-like format

- $e = 3$ exponent bits
- $f = 2$ fraction bits
- Bias is $2^{3-1}-1 = 3$



■ Notice how the distribution gets denser toward zero.

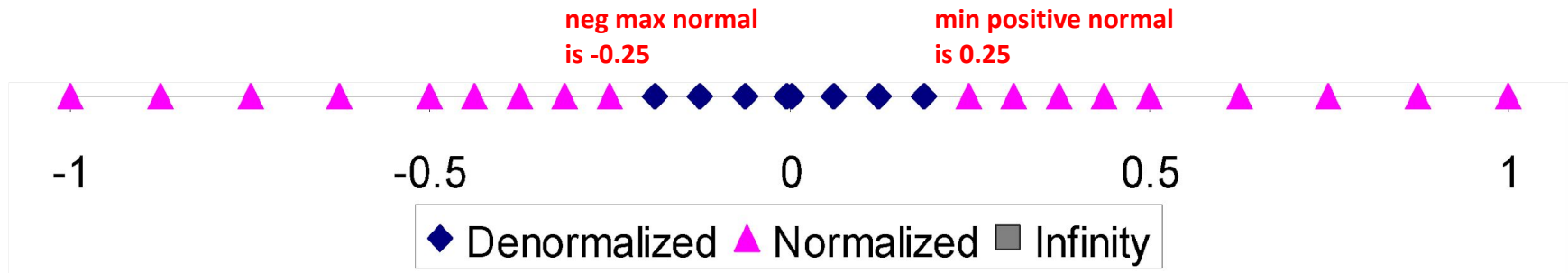
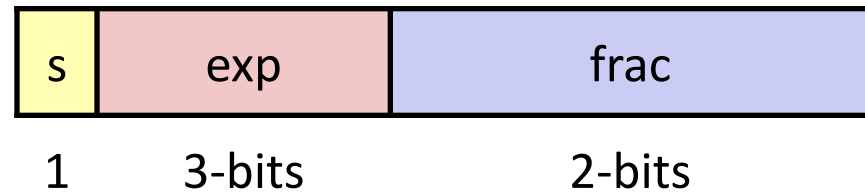


demo: ch2_6bits.c

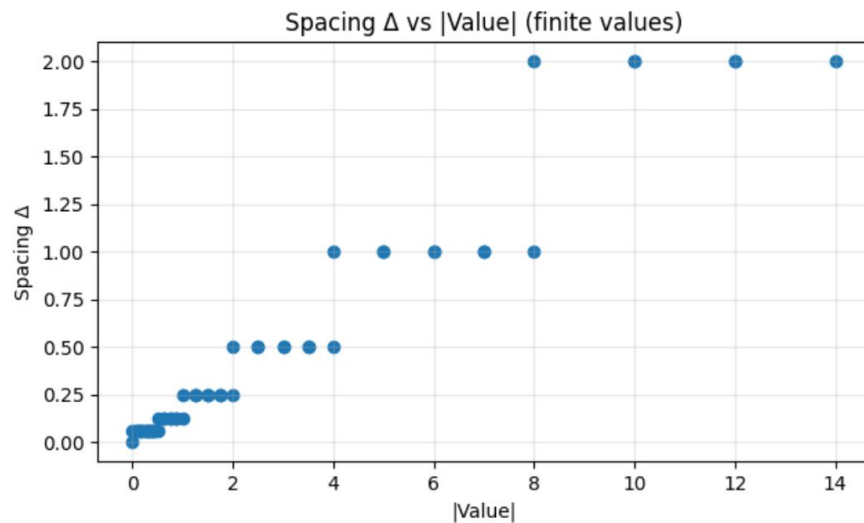
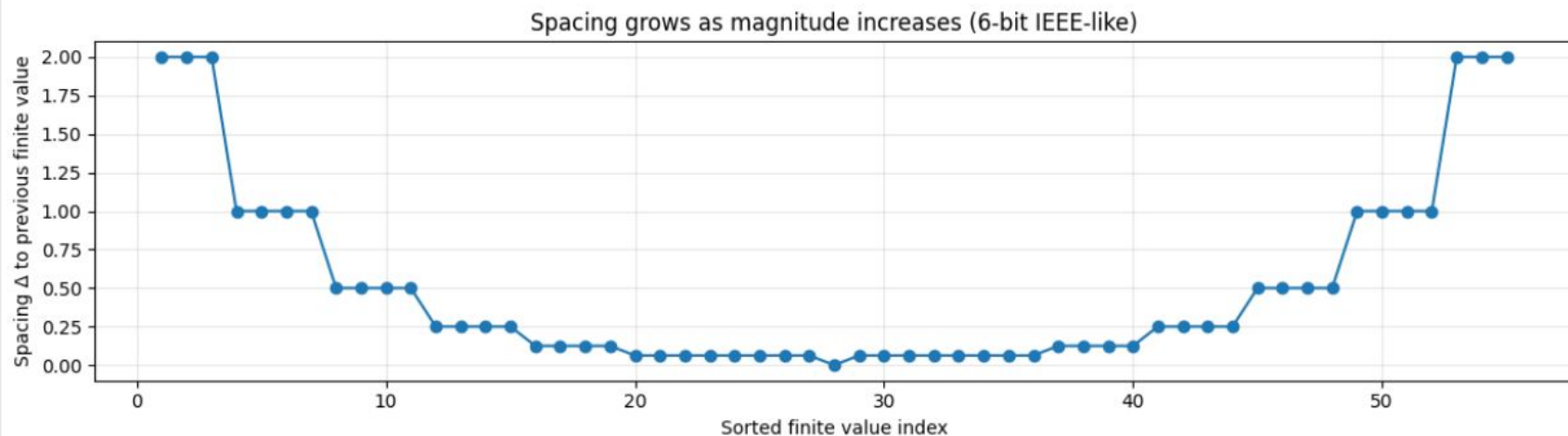
Distribution of Values (close-up view)

■ 6-bit IEEE-like format

- $e = 3$ exponent bits
- $f = 2$ fraction bits
- Bias is 3



More Visualization



Special Properties of the IEEE Encoding

■ FP Zero Same as Integer Zero

- All bits = 0

float f = 0.0;
unsigned u = 0;
Both are all-zero bits.

You can (almost) compare floats by comparing their bit patterns as unsigned integers.

■ Can (Almost) Use Unsigned Integer Comparison

- Must first compare sign bits $0x00000000 \neq 0x80000000$
- Must consider $-0 = 0$
- NaNs problematic NaN compared to anything $\rightarrow$ false
 - Will be greater than any other values
 - What should comparison yield?
- Otherwise OK
 - Denorm vs. normalized
 - Normalized vs. infinity

If signs differ:

negative < positive

If same sign:

- compare remaining bits
- reverse comparison for negatives

demo: ch2_float_specials.c

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- **Rounding, addition, multiplication**
- Floating point in C
- Summary

Floating Point Operations: Basic Idea

- $x +_f y = \text{Round}(x + y)$

- $x \times_f y = \text{Round}(x \times y)$

- **Basic idea**

- First **compute exact result**
- Make it fit into desired precision
 - Possibly overflow if exponent too large
 - Possibly **round to fit into `frac`**

Floating point numbers are approximations.

demo: `ch2_float_add.c`

Rounding

Round(exact result)

■ Rounding Modes (illustrate with \$ rounding)

■ \$1.40 \$1.60 \$1.50 \$2.50 -\$1.50

- Towards zero \$1 \$1 \$1 \$2 -\$1 **truncation, Always chop off decimals**
- Round down ($-\infty$) \$1 \$1 \$1 \$2 -\$2 **Always go to the smaller number.**
- Round up ($+\infty$) \$2 \$2 \$2 \$3 -\$1 **Always go to the larger number.**
- Nearest Even (default) \$1 \$2 \$2 \$2 -\$2 **If exactly halfway, ties to even**

Closer Look at Round-To-Even

Half go up, half go down.

Statistically unbiased.

■ Default Rounding Mode

- Hard to get any other kind without dropping into assembly
- All others are statistically biased
 - Sum of set of positive numbers will consistently be over- or under-estimated

■ Applying to Other Decimal Places / Bit Positions

- When exactly halfway between two possible values
 - Round so that least significant digit is **even**
- E.g., round to nearest hundredth

Make the last kept digit EVEN

7.8949999 7.89(Less than half way)

7.8950001 7.90(Greater than half way)

7.8950000 7.90(Half way—round up)

7.8850000 7.88(Half way—round down)

7.88 and 7.89
halfway: 7.8850000

Rounding Binary Numbers

■ Binary Fractional Numbers

- “Even” when least significant bit is 0
- “Half way” when bits to right of rounding position = $100..._2$

■ Examples

- Round to **nearest $1/4$** (2 bits right of binary point)

Value	Binary	Rounded	Action	Rounded Value
$2 \frac{3}{32}$	$10.00\textcolor{red}{011}_2$	10.00_2	(<1/2—down)	2
$2 \frac{3}{16}$	$10.00\textcolor{red}{110}_2$	10.01_2	(>1/2—up)	$2 \frac{1}{4}$
$2 \frac{7}{8}$	$10.11\textcolor{red}{100}_2$	11.00_2	(1/2—up)	3
$2 \frac{5}{8}$	$10.10\textcolor{red}{100}_2$	10.10_2	(1/2—down)	$2 \frac{1}{2}$

FP Multiplication

$$\blacksquare (-1)^{s1} M1 2^{E1} \times (-1)^{s2} M2 2^{E2}$$

$$\blacksquare \text{ Exact Result: } (-1)^s M 2^E$$

- Sign s : $s1 \wedge s2$
- Significand M : $M1 \times M2$
- Exponent E : $E1 + E2$

XOR rule:

same signs $\rightarrow$ positive

different signs $\rightarrow$ negative

Fixing

- If $M \geq 2$, shift M right, increment E
- If E out of range, overflow
- Round M to fit **frac** precision

1. XOR signs
2. Multiply significands
3. Add exponents
4. Normalize
5. Round
6. Check overflow

Implementation

- Biggest chore is multiplying significands

Floating Point Addition

$$\blacksquare (-1)^{s1} M1 2^{E1} + (-1)^{s2} M2 2^{E2}$$

- Assume $E1 > E2$

$$\blacksquare \text{ Exact Result: } (-1)^s M 2^E$$

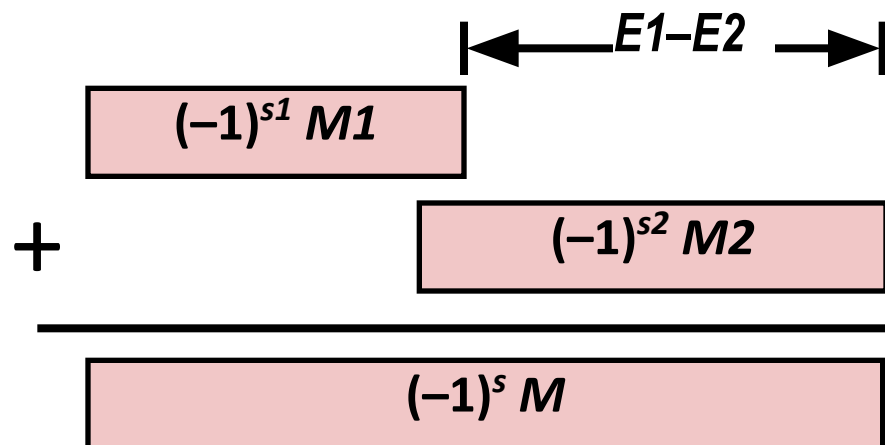
- Sign s , significand M :
 - Result of signed align & add
- Exponent E : $E1$

Fixing

- If $M \geq 2$, shift M right, increment E
- if $M < 1$, shift M left k positions, decrement E by k
- Overflow if E out of range
- Round M to fit **frac** precision

powers of 2 must match first.

Get binary points lined up



$$\begin{array}{r} 1.23 \times 10^5 \\ + 4.56 \times 10^3 \end{array} \longrightarrow \begin{array}{r} 1.23 \times 10^5 \\ + 0.0456 \times 10^5 \end{array}$$

Mathematical Properties of FP Add

■ Compare to those of Abelian Group

- Closed under addition?

Yes

If $a, b \in \mathbb{R} \rightarrow a + b \in \mathbb{R}$
Always closed.

- But may generate infinity or NaN

- Commutative?

Yes

$$a + b = b + a$$

- Associative?

No

$$(a + b) + c \neq a + (b + c)$$

- Overflow and inexactness of rounding

$$(3.14 + 1e10) - 1e10 = 0, \quad 3.14 + (1e10 - 1e10) = 3.14$$

- 0 is additive identity?

Yes

$$a + 0 = a$$

- Every element has additive inverse?

Almost

Almost every number has $-a$.

- Yes, except for infinities & NaNs

■ Monotonicity

- $a \geq b \Rightarrow a + c \geq b + c$

Almost

- Except for infinities & NaNs

Mathematical Properties of FP Mult

■ Compare to Commutative Ring

- Closed under multiplication?

Yes If $a, b \in \mathbb{R} \rightarrow a * b \in \mathbb{R}$
 Always closed.
- But may generate infinity or NaN
- Multiplication Commutative?

Yes $a * b = b * a$
- Multiplication is Associative?

No $(a * b) * c \neq a * (b * c)$

 - Possibility of overflow, inexactness of rounding
 - Ex: $(1e20 * 1e20) * 1e-20 = \text{inf}$, $1e20 * (1e20 * 1e-20) = 1e20$
- 1 is multiplicative identity?

Yes $a * 1 = a$
- Multiplication distributes over addition?

No $a * (b - c) \neq a * b - a * c$

 - Possibility of overflow, inexactness of rounding
 - $1e20 * (1e20 - 1e20) = 0.0$, $1e20 * 1e20 - 1e20 * 1e20 = \text{NaN}$

■ Monotonicity

- $a \geq b \ \& \ c \geq 0 \Rightarrow a * c \geq b * c$?

Almost

- Except for infinities & NaNs

Today: Floating Point

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- **Floating point in C**
- Summary

Floating Point in C

■ C Guarantees Two Levels

- `float` single precision: 2 bits, ~7 decimal digits precision
- `double` double precision 64 bits, ~15–16 decimal digits precision

■ Conversions/Casting

- Casting between `int`, `float`, and `double` changes bit representation

▪ `double/float` → `int`

- **Truncates** fractional part
- Like rounding toward zero

(int) 3.9 = 3
 (int)-3.9 = -3
 (int) 3.9f = 3
 (int)-3.9f = -3

- Not defined when out of range or NaN: Generally sets to TMin

▪ `int` → `double`

- Exact conversion, as long as **int has ≤ 53** bit word size

▪ `int` → `float`

- Will round according to rounding mode

Floating Point Puzzles

■ For each of the following C expressions, either:

- Argue that it is true for all argument values
- Explain why not true

```
int x = ...; 32bits
float f = ...; 32bits
double d = ...; 64bits
```

Assume neither
d nor f is NaN

- `x == (int)(float) x` **NOT always true**
int $\rightarrow$ float may lose precision
then (float $\rightarrow$ int) truncates
- `x == (int)(double) x` **TRUE (on normal systems)**
double has 53 bits precision
int usually ≤ 32 bits
- `f == (float)(double) f` **TRUE**
- `d == (double)(float) d` **NOT always true**
- `f == -(-f);` **TRUE (assuming not NaN)**
- `2/3 == 2/3.0` **FALSE**. $2/3 \rightarrow$ integer division $\rightarrow 0$
- `d < 0.0  $\Rightarrow$  ((d*2) < 0.0)` **NOT always true, overflow**
- `d > f  $\Rightarrow$  -f > -d` **TRUE (assuming no NaN)**
- `d * d >= 0.0` **NOT always true, overflow**
- `(d+f) - d == f` **NOT always true,**

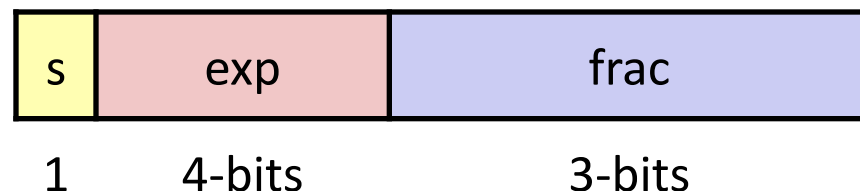
Summary

- IEEE Floating Point has clear mathematical properties
- Represents numbers of form $M \times 2^E$
- One can reason about operations independent of implementation
 - As if computed with perfect precision and then rounded
- Not the same as real arithmetic
 - Violates associativity/distributivity
 - Makes life difficult for compilers & serious numerical applications programmers

Creating Floating Point Number

■ Steps

- Normalize to have leading 1
- Round to fit within fraction
- Postnormalize to deal with effects of rounding



■ Case Study

- Convert 8-bit unsigned numbers to tiny floating point format

Example Numbers

128 **10000000**

$10000000_2 = 1.0000000 \times 2^7$
1.000

$E = 7$
 $\text{exp_field} = E + \text{bias}$
 $= 7 + 7 = 14$
 $14 = 1110_2$

$S = 0$
 $\text{exp} = 1110$ **0 1100 000**
 $\text{frac} = 000$

15 **00001101**

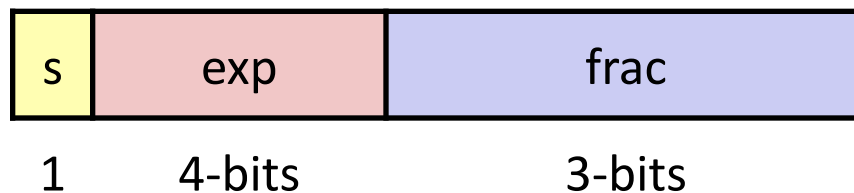
33 **00010001**

35 **00010011**

138 **10001010**

63 **00111111**

Normalize



■ Requirement

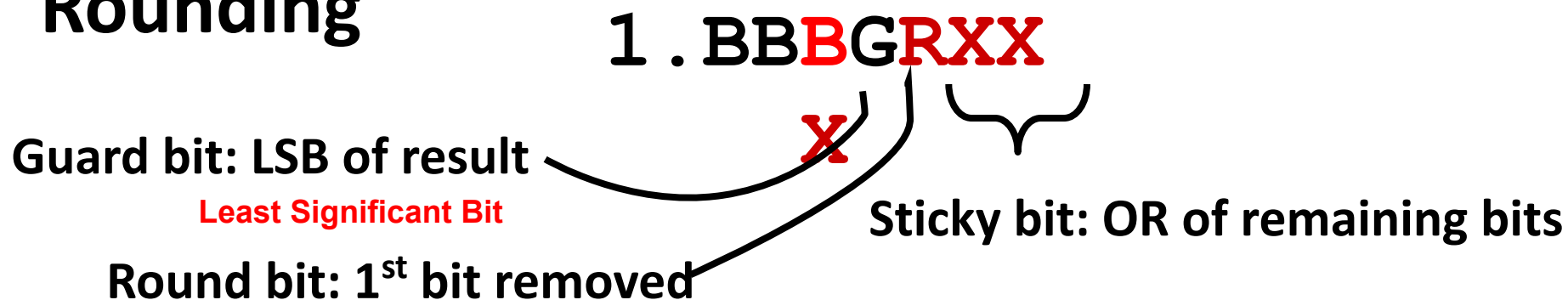
- Set binary point so that numbers of form **1.xxxxx**
- Adjust all to have leading one
 - Decrement exponent as shift left

It gives **one unique representation**
 It maximizes precision
 It lets us drop the leading 1 (implicit bit)

Value Binary Fraction Exponent

128	10000000	1.0000000	7
15	00001101	1.1010000	3
17	00010001	1.0001000	4
19	00010011	1.0011000	4
138	10001010	1.0001010	7
63	00111111	1.1111100	5

Rounding



■ Round up conditions

- Round = 1, Sticky = 1 $\rightarrow$ > 0.5
- Guard = 1, Round = 1, Sticky = 0 $\rightarrow$ Round to even

Value *Fraction* *GRS* *Incr?* *Rounded*

128 1.000**0000** 000 N 1.000

15 1.101**0000** 100 N 1.101

17 1.000**1000** 010 N 1.000

19 1.001**1000** 110 Y 1.010

138 1.000**1010** 011 Y 1.001

63 1.111**1100** 111 Y 10.000

$$1.001 + 0.001 = 1.010$$

G=1 R=0 rest=10... $\Rightarrow$ sticky=1 $\Rightarrow$ Yes

G=1 R=1 rest=00... $\Rightarrow$ tie/over-half
depending on sticky; here it rounds up $\Rightarrow$ Yes

Postnormalize

■ Issue

- Rounding may have caused overflow
- Handle by shifting right once & incrementing exponent

<i>ValueRounded</i>		<i>Exp Adjusted</i>	<i>Result</i>
128	1.0007	128	0 1100 000
15	1.1013	15	
17	1.0004	16	
19	1.0104	20	
138	1.0017	134	
63	10.0005	1.000/6	64

Interesting Numbers

{single, double}

Description exp frac Numeric Value

- **Zero** 00...00 00...00 0.0
 - **Smallest Pos. Denorm.** 00...00 00...01 $2^{-\{23,52\}} \times 2^{-\{126,1022\}}$
 - Single $\approx 1.4 \times 10^{-45}$
 - Double $\approx 4.9 \times 10^{-324}$
 - **Largest Denormalized** 00...00 11...11 $(1.0 - \epsilon) \times 2^{-\{126,1022\}}$
 - Single $\approx 1.18 \times 10^{-38}$
 - Double $\approx 2.2 \times 10^{-308}$
 - **Smallest Pos. Normalized** 00...01 00...00 $1.0 \times 2^{-\{126,1022\}}$
 - Just larger than largest denormalized
 - **One** 01...11 00...00 1.0
 - **Largest Normalized** 11...10 11...11 $(2.0 - \epsilon) \times 2^{\{127,1023\}}$
 - Single $\approx 3.4 \times 10^{38}$
 - Double $\approx 1.8 \times 10^{308}$
- 0 ---- denormals ---- smallest normal ---- larger normals